

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Surendar MG	Reg. No:	19A2521374
Date:	14/08/2026	Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.
- 6. Commit History Analysis**
 - Present the commit history of the project.

- Explain how commit messages follow good version control practices and contribute to project maintenance.
- 7. Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
- 8. Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
- 9. Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
- 10. Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
- 11. Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
- 12. Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.

Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

GIT & DOCKER TECHNICAL ASSIGNMENT

Smart Forest Fire Detection and Environmental Monitoring System

Report Purpose

This report applies Git version control and Docker containerization to the assigned Smart Forest Fire Detection and Environmental Monitoring System. It follows the 12 questions in Annexure A and is structured to provide the problem analysis, repository design, Git workflow, Docker environment, deployment/testing approach, benefits, challenges and future improvements required by the assessment.

1. Problem Analysis

1.1 Industry context and problem

Forest fires can spread rapidly when dry vegetation, high temperature, low humidity and wind combine with an ignition source. Conventional manual observation can be slow, while satellite observation may not provide sufficiently local or continuous measurements for every forest zone. The proposed platform combines IoT environmental sensing, AI-based risk analysis, satellite and drone observations, cloud analytics and emergency notifications to support earlier detection and better response.

1.2 Project objectives

The system objectives are to monitor environmental conditions; detect fires at an early stage; predict fire risk using AI; integrate drone and satellite monitoring; generate emergency alerts; support firefighting operations; produce environmental analytics reports; and improve forest conservation.

1.3 Functional requirements

FR1	Collect temperature, humidity, smoke and wind-speed readings from field IoT sensors.
FR2	Receive, validate and timestamp sensor data at an edge/gateway layer.
FR3	Calculate a fire-risk score from current and historical environmental data.
FR4	Detect probable fire incidents and classify alert severity.
FR5	Generate immediate alerts for authorized emergency personnel.
FR6	Ingest drone observations and satellite/fire-detection data.
FR7	Display live environmental and incident information on a monitoring dashboard.
FR8	Maintain incident history, sensor history and audit records.
FR9	Provide analytics and periodic environmental/fire-risk reports.
FR10	Support authorized responders in viewing incidents and response status.

1.4 Expected outcomes

Early fire detection, reduced environmental damage, improved emergency response, better forest monitoring, enhanced biodiversity protection, reduced wildfire spread, data-driven forest management and sustainable environmental conservation.

1.5 Why Git and Docker are relevant

The system contains several software components—dashboard, API, AI analytics, sensor ingestion and data services. Git provides traceable version history and parallel development, while Docker packages each service with its runtime dependencies. Together they reduce environment-related inconsistencies and make the proposed multi-service platform easier to reproduce and deploy.

2. Project Structure Design

2.1 Proposed repository structure

```
smart-forest-fire-system/
├── frontend/                # Monitoring dashboard
│   ├── src/
│   │   ├── public/
│   │   └── package.json
│   └── backend/            # REST API and alert orchestration
│       ├── src/
│       └── package.json
├── ai-service/              # Fire-risk / anomaly inference
│   ├── app/
│   ├── models/
│   └── requirements.txt
├── sensor-gateway/          # IoT/MQTT ingestion and validation
│   ├── src/
│   └── requirements.txt
├── database/
│   └── init.sql
├── docker/
│   ├── nginx.conf
│   └── mosquitto.conf
├── tests/
│   ├── api/
│   ├── ai/
│   └── integration/
├── docs/
│   ├── architecture/
│   └── screenshots/
├── .env.example
├── .gitignore
├── Dockerfile
├── docker-compose.yml
└── README.md
```

2.2 Folder/file justification

Path	Purpose
frontend/	User-facing monitoring dashboard for live sensor status, maps, alerts and reports.
backend/	Business logic, authentication/authorization, REST APIs and incident orchestration.
ai-service/	Model inference, feature preparation and fire-risk scoring.
sensor-gateway/	MQTT/IoT message intake, validation, buffering and forwarding.
database/	Schema and initialization scripts for persistent application data.
docker/	Infrastructure configuration such as reverse proxy and MQTT broker settings.
tests/	Unit, integration and API test assets.
docs/	Architecture diagrams, setup instructions and evidence screenshots.
.gitignore	Prevents secrets, generated files and local environment artifacts from being committed.
docker-compose.yml	Defines services, networks, volumes, environment variables and dependencies.
README.md	Explains setup, commands, architecture, usage and contribution workflow.

Figure 1. Repository organization

The structure separates application concerns and infrastructure configuration. This supports maintainability because a change in the AI service, for example, can be reviewed and tested without mixing its files with the frontend or deployment configuration.

3. Git Repository Initialization

3.1 Initialization procedure

```
mkdir smart-forest-fire-system
cd smart-forest-fire-system

git init
git branch -M main

git add .
git commit -m "Initial project structure"

git remote add origin <YOUR_GITHUB_REPOSITORY_URL>
git push -u origin main
```

The working directory becomes a Git repository after **git init**. The main branch represents the stable baseline. The first commit establishes a reproducible starting point. The remote connects the local repository with the GitHub repository used for collaboration.

3.2 Essential Git files and concepts

Item	Purpose
.git/	Internal Git database containing commits, references, objects and configuration.
.gitignore	Lists files/folders Git should not track, such as .env, node_modules and generated logs.
README.md	Project overview, setup steps, architecture, commands and contribution guidance.
Branches	Independent lines of development used to isolate features and fixes.
Remote origin	Named reference to the shared GitHub repository.

3.3 Repository hygiene

Secrets such as API keys, cloud credentials and database passwords must never be committed. Use environment variables and an .env.example file containing only placeholder names. Large generated model artifacts should be stored using an appropriate artifact/model registry rather than repeatedly committed into Git history.

4. Git Branching Strategy

4.1 Selected strategy: Main + Development + Feature + Hotfix

```
main
  ■
  ■■■ development
    ■■■ feature/sensor-ingestion
    ■■■ feature/fire-risk-model
    ■■■ feature/alert-service
    ■■■ feature/dashboard

Emergency production correction:
main ■■ hotfix/critical-alert-fix ■■> main + development
```

The project uses a lightweight Git-flow-inspired model. **main** contains stable releases. **development** integrates completed features before a release. Each feature is isolated in a feature branch. A hotfix branch is created directly from main when a critical production defect requires urgent correction.

4.2 Example branch commands

```
git checkout development
git pull origin development

git checkout -b feature/sensor-ingestion
# implement and test the feature

git add .
git commit -m "Add IoT sensor ingestion service"
git push -u origin feature/sensor-ingestion
```

4.3 Pull request workflow

A feature branch is pushed to GitHub and submitted as a pull request into development. Another team member reviews the changes, checks automated tests, comments on defects, and approves the PR when the acceptance criteria are met. The feature is then merged. Release-ready development is merged into main after final validation.

Why this is suitable: The system has independent components and features that can be developed in parallel, while main remains protected from incomplete changes.

5. Version Control Workflow

5.1 Standard feature workflow

```
git status
git checkout -b feature/alert-service

# edit files
git add backend/ ai-service/
git commit -m "Add emergency alert orchestration"

git fetch origin
git rebase origin/development

# resolve conflicts if Git reports them
git add <resolved-file>
git rebase --continue

git push -u origin feature/alert-service
```

The workflow keeps changes small and traceable. **git status** identifies the working-tree state; **git add** stages intended changes; **git commit** records a logical unit of work; **git fetch** updates remote references; and rebase/merge integrates the latest shared changes.

5.2 Merge conflict resolution

```
git checkout feature/alert-service
git fetch origin
git merge origin/development

# If a conflict occurs:
git status
# Open each conflicted file and remove:
<<<<<< HEAD
your branch version
=====
development version
>>>>>> origin/development

# Keep the correct combined implementation, then:
git add <file>
git commit -m "Resolve merge conflict with development"
git push
```

A conflict occurs when two branches modify overlapping parts of the same file in incompatible ways. The developer must understand both versions, combine them safely, run tests, stage the resolved files and commit the resolution. Conflict resolution should never be performed by blindly selecting one side.

Evidence to capture: terminal showing branch creation, commit, push, pull request, conflict (if intentionally demonstrated), resolution, and final synchronization.

6. Commit History Analysis

6.1 Recommended commit sequence

Order	Example commit message	Purpose
1	Initial project structure	Create folders, README and baseline configuration.
2	Add IoT sensor ingestion service	Introduce gateway/MQTT data handling.
3	Implement fire risk scoring API	Add AI inference endpoint and validation.
4	Add emergency alert service	Connect severity classification to notifications.
5	Add monitoring dashboard	Display sensor and incident information.
6	Containerize application services	Add Dockerfiles and Compose configuration.
7	Add integration tests	Validate service-to-service communication.

6.2 Commit message quality

Good commit messages are concise, specific and written around one logical change. Messages such as "Fix alert threshold validation" are more useful than "changes" because maintainers can understand the intent from the history. Commits should avoid mixing unrelated features, formatting changes and large generated artifacts.

```
git log --oneline --graph --decorate --all
git log --stat
git show <commit-id>
```

These commands provide evidence of history, branch relationships, file-level change statistics and the exact content introduced by an individual commit. For the submitted experiment, a screenshot of the actual project history should be captured after completing the workflow.

7. Docker Environment Design

7.1 Why Docker is suitable

The forest monitoring platform is naturally multi-service: a browser dashboard, API server, AI inference service, sensor gateway, message broker and database have different runtime dependencies. Docker packages these dependencies into isolated containers so development, testing and deployment can use consistent environments.

7.2 Proposed container architecture

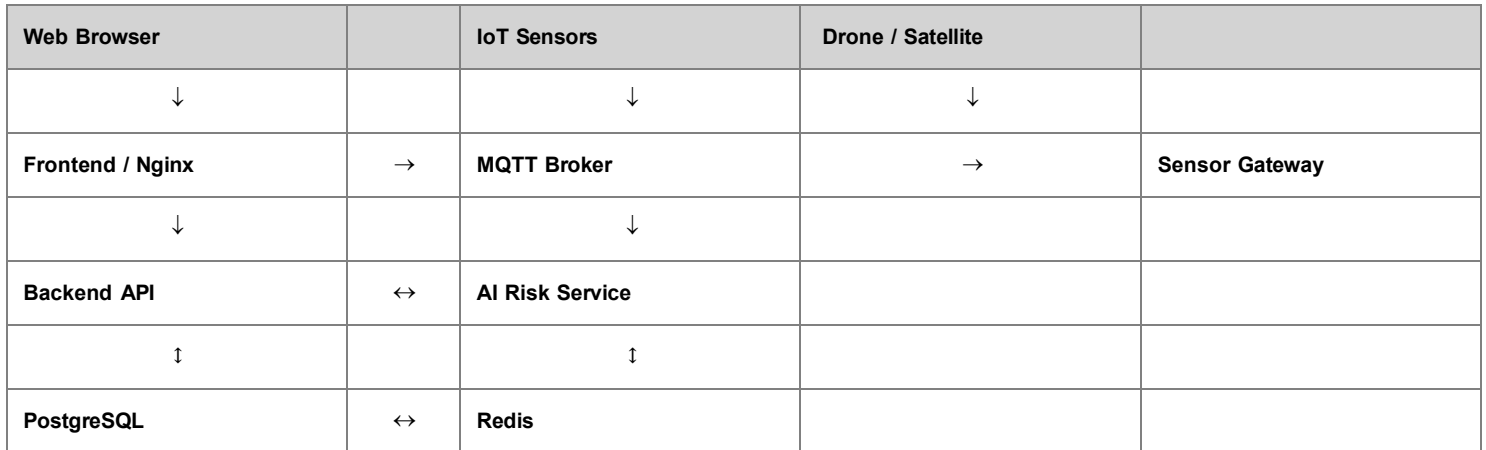


Figure 2. Proposed containerized service architecture

7.3 Main services

Service	Responsibility
frontend	Serves monitoring UI and dashboard assets.
backend	REST API, authentication, incidents, reports and alert orchestration.
ai-service	Feature processing and fire-risk inference.
sensor-gateway	Consumes MQTT sensor messages, validates readings and forwards data.
mqtt	Lightweight publish/subscribe transport between sensor gateways and services.
postgres	Persistent users, sensors, readings, incidents and audit records.
redis	Short-lived cache, queues/rate limiting and transient alert state.

8. Dockerfile Development

8.1 Example backend Dockerfile

```
# backend/Dockerfile
FROM node:22-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

ENV NODE_ENV=production
EXPOSE 8080

USER node
CMD ["node", "src/server.js"]
```

8.2 Instruction-by-instruction explanation

Instruction	Purpose
FROM node:22-alpine	Uses a small Node.js runtime image as the base.
WORKDIR /app	Sets the container working directory.
COPY package*.json ./	Copies dependency manifests before source code to improve build-cache reuse.
RUN npm ci --omit=dev	Installs production dependencies reproducibly from the lock file.
COPY . .	Copies the application source into the image.
ENV NODE_ENV=production	Sets the runtime environment.
EXPOSE 8080	Documents the application port used inside the container.
USER node	Avoids running the application as root.
CMD ...	Starts the backend service when the container runs.

8.3 Build and run

```
docker build -t forest-backend:1.0 ./backend
docker run --rm -p 8080:8080 --env-file .env forest-backend:1.0
```

In a production implementation, images should be versioned using immutable tags or image digests, health checks should be defined, and secrets should be injected at runtime rather than baked into the image.

Additional Dockerfiles: The frontend can use a multi-stage Node/Nginx build, while the AI service can use a Python slim image with a pinned requirements file. The sensor gateway can use a small Python or Node runtime depending on implementation.

9. Docker Compose Design

9.1 Example docker-compose.yml

```
services:
  frontend:
    build: ./frontend
    ports:
      - "80:80"
    depends_on:
      - backend

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://forest:forest@postgres:5432/forest
      REDIS_URL: redis://redis:6379
      AI_URL: http://ai-service:8000
    ports:
      - "8080:8080"
    depends_on:
      - postgres
      - redis
      - ai-service

  ai-service:
    build: ./ai-service
    expose:
      - "8000"

  sensor-gateway:
    build: ./sensor-gateway
    environment:
      MQTT_BROKER: mqtt
    depends_on:
      - mqtt

  mqtt:
    image: eclipse-mosquitto:2
    volumes:
      - mqtt_data:/mosquitto/data
      - ./docker/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro

  postgres:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: forest
      POSTGRES_USER: forest
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:8-alpine

volumes:
  mqtt_data:
  postgres_data:
```

9.2 Compose design explanation

Element	Application in this project
services	Defines independently deployable application and infrastructure components.
ports	Publishes only the ports that must be reachable from the host.
depends_on	Expresses startup dependencies; application-level health checks should also be used for readiness.
environment	Injects service endpoints and runtime configuration.
volumes	Persists PostgreSQL and MQTT data beyond container recreation.

network	Compose provides an internal network so services can address each other by service name.
secrets	Production deployment should use a secrets mechanism instead of hard-coded credentials.

9.3 Startup commands

```
docker compose config
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f backend
docker compose down
```

10. Container Deployment and Testing

10.1 Deployment procedure

```
# Validate configuration
docker compose config

# Build all images
docker compose build

# Start services in background
docker compose up -d

# Check service state
docker compose ps

# Inspect logs
docker compose logs --tail=100 backend
docker compose logs --tail=100 ai-service

# Test API from host
curl http://localhost:8080/health
```

10.2 Test cases

ID	Test	Expected result
T01	Start Compose stack	All required services become running/healthy.
T02	Open dashboard	Frontend responds and loads the monitoring interface.
T03	Send sample sensor message	Gateway accepts valid data and forwards it to the platform.
T04	Send invalid sensor value	Validation rejects or flags the reading without crashing the service.
T05	AI risk request	AI service returns a valid risk score/classification.
T06	High-risk condition	Backend creates an incident and invokes alert workflow.
T07	Database persistence	Sensor/incident records remain after application restart.
T08	Service restart	Affected service restarts without losing persistent database data.
T09	API health check	Health endpoint returns successful response when service is ready.
T10	Compose shutdown/start	Stack can be stopped and restarted reproducibly.

10.3 Required screenshots for submission

Capture actual evidence for: (1) GitHub repository structure, (2) git branch list, (3) commit history, (4) pull request/merge where used, (5) Dockerfile, (6) docker-compose.yml, (7) docker compose ps, (8) application running in the browser, (9) container logs, and (10) successful test/API response. These screenshots should be inserted in the final submitted version because the Annexure explicitly asks for screenshots and evidence.

11. Benefits, Challenges and Future Enhancements

11.1 Benefits of Git

Git provides distributed version control, traceability, branching, rollback and parallel development. For the forest platform, separate feature branches allow sensor, AI, alert and dashboard work to proceed without destabilizing the main branch. Pull requests provide a review point before integration.

11.2 Benefits of Docker

Docker provides reproducible runtime environments, service isolation and easier deployment. A developer does not need to manually install every service dependency on the host. Docker Compose also gives the student a simple way to start the complete multi-container stack.

11.3 Challenges

Potential challenges include sensor connectivity failures, unstable network links in remote forests, model drift, false alarms, large time-series datasets, container resource consumption, secret management, database backup, and coordination between multiple branches. AI predictions must also be treated as decision support rather than an unquestionable replacement for trained emergency personnel.

11.4 Improvements and future enhancements

Future versions can add Kubernetes-based orchestration, automated CI/CD pipelines, image scanning, centralized logging, model monitoring, edge inference for disconnected areas, digital maps, drone mission automation, satellite-data ingestion, role-based access control, encrypted MQTT, automated database backups, disaster recovery and stronger observability.

11.5 Rubric alignment

Rubric criterion	Where addressed in this report
Problem Analysis & Project Design (20%)	Sections 1–2: problem, objectives, requirements, outcomes and repository structure.
Git Repository & Branching Strategy (20%)	Sections 3–4: initialization, files, branches and workflow.
Version Control Workflow & Commit History (10%)	Sections 5–6: commands, conflicts, commits and history analysis.
Docker Implementation (20%)	Sections 7–9: container architecture, Dockerfile and Compose.
Deployment, Testing & Results (15%)	Section 10: deployment commands, test cases and evidence checklist.
Documentation, Challenges & Future Enhancements	Section 11: benefits, challenges, improvements and submission evidence.

Conclusion

The proposed Git and Docker workflow gives the Smart Forest Fire Detection and Environmental Monitoring System a structured engineering foundation. Git controls source evolution and collaboration, while Docker standardizes the multi-service runtime. The combination supports reproducibility, maintainability and a clear path from local development to automated deployment.

12. Submission Checklist and References

12.1 Final practical checklist

✓	Problem statement, objectives, functional requirements and expected outcomes documented.
✓	Repository structure created and explained.
✓	Git repository initialized and connected to GitHub.
✓	Main/development/feature branching strategy documented.
✓	Meaningful commits created with clear messages.
✓	Branch merge/pull request workflow demonstrated.
✓	Conflict resolution demonstrated if required by the instructor.
✓	Dockerfile(s) created and explained.
✓	docker-compose.yml created with services, networking, volumes and environment variables.
✓	Containers built, started, inspected and tested.
✓	Actual screenshots inserted before final submission.
✓	GitHub repository link added to the report.

12.2 Useful Git commands

```
git status
git branch
git switch -c feature/name
git add .
git commit -m "Meaningful message"
git fetch origin
git pull
git push
git log --oneline --graph --all
git merge <branch>
git remote -v
```

12.3 Useful Docker commands

```
docker --version
docker compose version
docker compose config
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f
docker compose exec <service> sh
docker compose down
```

References

1. Git Documentation – Git SCM documentation and command reference.
2. Docker Documentation – Docker Engine and Docker Compose documentation.
3. Docker Compose Specification – service, network, volume and configuration concepts.
4. The assigned problem statement supplied with this assessment: Smart Forest Fire Detection and Environmental Monitoring System.

End of Technical Report

Student: Surendar MG | Register No.: 192521374 | Date: 14/08/2026